

Documentacion de la Entrega Semana 3

Carga de Footprints de Edificios y Procedimientos de Administracion en MongoDB

Proyecto-DBA

25 de mayo de 2026

1. Resumen

La entrega de la semana 3 corresponde a la integracion de huellas de edificios en MongoDB a partir de dos fuentes abiertas: Microsoft Building Footprints y Google Open Buildings. En esta version del proyecto ya se dispone de un flujo funcional para localizar el archivo fuente, leer la informacion geoespacial, transformarla a una estructura compatible con MongoDB y cargarla en colecciones separadas por dataset.

Ademas del proceso de ingesta, esta etapa involucra procedimientos propios de administracion de bases de datos, especialmente en la preparacion de colecciones, normalizacion del modelo documental, indexacion geoespacial y control operativo del flujo de carga.

2. Objetivo de la semana 3

El objetivo es dejar almacenadas en MongoDB las huellas de edificios provenientes de Microsoft y Google, con geometria en formato GeoJSON y metadatos minimos que permitan su uso posterior en consultas espaciales, joins con municipios PDET y agregaciones territoriales.

3. Descripcion general de la arquitectura

El flujo implementado durante la semana 3 se apoya en tres modulos principales:

- `main.py`: orquesta la ejecucion del sistema y presenta el menu principal al usuario.
- `download_manager.py`: verifica la disponibilidad de archivos locales y, si es necesario, permite descargar o registrar manualmente los datasets de edificios.
- `load_buildings.py`: realiza la lectura geoespacial, la transformacion al modelo documental y la carga final en MongoDB.

Desde la perspectiva de administracion de base de datos, el sistema separa los datos por fuente en colecciones independientes, lo que facilita la gestion de indices, la trazabilidad de la informacion y la futura comparacion entre datasets.

4. Flujo operativo del sistema

El procedimiento actual de ejecucion puede resumirse en los siguientes pasos:

1. El usuario ejecuta `python main.py`.
2. El sistema verifica que existan los archivos base del proyecto, incluyendo el Excel PDET y el shapefile municipal.
3. Desde el menu principal se selecciona la opcion de ingesta para Microsoft o Google.
4. El modulo `download_manager.py` comprueba si el archivo del dataset existe localmente.
5. Si el archivo no esta disponible, el sistema ofrece intentar la descarga o pedir una ruta local.
6. Se solicita la configuracion de conexion a MongoDB.
7. Se confirma la ejecucion de la carga.
8. El script `load_buildings.py` abre el archivo geoespacial con GeoPandas.
9. Se valida el sistema de coordenadas y se reproyecta a `EPSG:4326` cuando es necesario.
10. Se crea el indice geoespacial `2dsphere` en la coleccion correspondiente.
11. Se transforman las geometrias a GeoJSON y se construyen los documentos.
12. Los documentos se insertan de forma masiva en MongoDB.

5. Descripcion del codigo

5.1. Menu principal y control de ejecucion

El archivo `main.py` centraliza la interaccion con el usuario. Su funcion es actuar como punto de entrada del sistema y enrutar cada opcion hacia el procedimiento adecuado. Para semana 3, las opciones relevantes son la 3 y la 4, correspondientes a la carga de huellas de edificios de Microsoft y Google.

```
print("\n1. Cargar datos en MongoDB")
print("2. Verificar municipios PDET")
print("3. Ingestar huellas de edificios MICROSOFT")
print("4. Ingestar huellas de edificios GOOGLE")
print("5. Salir")
```

Cuando el usuario elige una de estas opciones, el programa no carga directamente el dataset. Primero verifica el archivo fuente, obtiene la configuracion de MongoDB y pide confirmacion explicita antes de ejecutar la insercion. Ese comportamiento es importante desde la administracion operativa porque evita cargas accidentales y separa claramente la etapa de configuracion de la etapa de escritura sobre la base de datos.

5.2. Gestion y disponibilidad de archivos

El modulo `download_manager.py` implementa funciones auxiliares para administracion de insumos. En el caso de los datasets de edificios, la funcion `check_and_download_buildings(dataset_type)` define la politica actual de disponibilidad del archivo.

```
if dataset_type == "microsoft":
    url = MICROSOFT_BUILDINGS_LINK
    filename = MS_BUILDINGS_FILE
else:
    url = GOOGLE_BUILDINGS_LINK
    filename = GOOG_BUILDINGS_FILE
```

El procedimiento asociado es:

- verificar si el archivo ya existe en el directorio local;
- en caso negativo, ofrecer una descarga remota;
- si no se descarga, solicitar manualmente una ruta local.

Este enfoque cumple una funcion administrativa importante: desacopla la logica de base de datos de la administracion fisica de archivos, lo cual permite usar datasets pesados sin obligar a mantenerlos siempre dentro del repositorio.

5.3. Conexion a MongoDB

La configuracion de conexion se realiza de manera interactiva. El sistema acepta una direccion IP personalizada o utiliza por defecto `mongodb://localhost:27017/`.

```
if mongo_ip == "":
    mongo_uri = "mongodb://localhost:27017/"
else:
    mongo_uri = f"mongodb://{mongo_ip}:27017/"
```

Desde la perspectiva DBA, esto permite ejecutar la misma aplicacion sobre una instancia local o una instancia remota sin modificar el codigo principal, lo cual es util para pruebas, validacion y futuras migraciones de ambiente.

5.4. Proceso de ingesta en MongoDB

La funcion principal de semana 3 es `run_buildings_ingestion(file_path, dataset_type)` definida en `load_buildings.py`. Esta funcion abre una conexion con MongoDB, selecciona la base `upme-project` y dirige la carga a una coleccion dependiente de la fuente.

```
client = MongoClient(MONGO_URI)
db = client[DB_NAME]
col_name = f"buildings_{dataset_type}"
collection = db[col_name]
```

Esto genera dos colecciones diferenciadas:

- `buildings_microsoft`

- `buildings_google`

La separacion por coleccion tiene varias ventajas administrativas:

- evita mezclar registros con estructuras distintas;
- simplifica la trazabilidad del origen de los datos;
- permite crear y ajustar indices de manera independiente;
- facilita comparar calidad, volumen o cobertura entre fuentes.

5.5. Lectura geoespacial y normalizacion

El archivo geoespacial es cargado con GeoPandas:

```
gdf = gpd.read_file(file_path)
```

Despues de la lectura, el sistema valida el CRS del dataset. Si no coincide con EPSG:4326, se hace reproyeccion a WGS84.

```
if gdf.crs != "EPSG:4326":  
    gdf = gdf.to_crs(epsg=4326)
```

Esta operacion es clave desde la administracion espacial de MongoDB porque el indice 2dsphere trabaja sobre geometria GeoJSON en coordenadas geograficas. Mantener un CRS consistente es un requisito para que las futuras consultas espaciales se comporten correctamente.

5.6. Transformacion al modelo documental

El sistema recorre cada fila del `GeoDataFrame`, descarta geometrias vacias y transforma el objeto geometrico de Shapely a una estructura compatible con GeoJSON.

```
geom = row['geometry']  
if geom is None or geom.is_empty:  
    continue  
  
doc = {  
    "geometry": geom.__geo_interface__,  
    "area_m2": round(float(area_m2), 2),  
    "dataset_source": dataset_type,  
    "ingested_at": datetime.now(timezone.utc)  
}
```

El modelo documental actual contiene:

- `geometry`: geometria del edificio en formato GeoJSON.
- `area_m2`: area estimada del footprint.
- `dataset_source`: identificador de la fuente.
- `ingested_at`: fecha y hora de insercion en UTC.

Para Google se incorpora un campo adicional:

```
if dataset_type == "google" and "confidence" in row:
    doc["confidence_score"] = float(row["confidence"])
```

Esto demuestra un rasgo importante del modelo NoSQL utilizado: el esquema es flexible y permite almacenar atributos propios de una fuente sin obligar a que todos los documentos tengan exactamente la misma estructura.

6. Procedimientos de administracion en base de datos

6.1. Creacion de colecciones por dominio

El sistema no crea manualmente las colecciones mediante comandos administrativos previos; en su lugar, MongoDB las materializa automaticamente cuando se inserta el primer documento. Aun asi, el diseño logico de administracion ya esta definido en el codigo:

- la base de datos operativa es `upme-project`;
- cada fuente de edificios tiene su propia coleccion;
- la separacion evita conflictos entre estructuras heterogeneas.

Este criterio facilita la administracion de almacenamiento y el control de operaciones futuras, como respaldos selectivos, consultas por fuente y mantenimiento de indices independientes.

6.2. Indexacion geoespacial

Uno de los procedimientos mas importantes de administracion implementados en esta semana es la creacion preventiva del indice geoespacial. La funcion `ensure_spatial_indexes` ejecuta:

```
db[col_name].create_index([("geometry", GEOSPHERE)])
```

Este indice corresponde al tipo `2dsphere` de MongoDB y permite:

- realizar consultas de interseccion espacial;
- soportar operadores como `$geoIntersects`, `$near` y `$geoWithin`;
- optimizar el costo de consultas sobre geometrias complejas.

Desde una perspectiva DBA, crear el indice antes o durante la carga asegura que la coleccion quede preparada para la siguiente fase del proyecto, en la cual se realizaran joins espaciales entre edificios y municipios PDET.

6.3. Normalizacion espacial para compatibilidad con el indice

La indexacion geoespacial por si sola no es suficiente. Para que el indice funcione correctamente, los documentos deben cumplir reglas minimas de integridad espacial:

- la geometria debe almacenarse como GeoJSON;
- las coordenadas deben estar en WGS84;
- las geometrias vacias no deben insertarse.

El codigo actual ya aplica estas tres decisiones durante la ingesta. Por tanto, la administracion espacial de la base no se limita a crear indices, sino que empieza desde la preparacion de los datos antes de insertarlos.

6.4. Insercion masiva de documentos

Una vez transformados los registros, el sistema ejecuta una insercion masiva:

```
collection.insert_many(documents)
```

Desde el punto de vista administrativo, esta decision busca reducir el costo de operaciones de escritura individuales y mejorar el rendimiento frente a cargas de mayor volumen. En lugar de insertar un documento por vez, la aplicacion agrupa todos los registros contruidos y los envia en una sola operacion logica a MongoDB.

6.5. Trazabilidad temporal

Cada documento incluye el campo `ingested_at`, calculado con fecha y hora UTC:

```
"ingested_at": datetime.now(timezone.utc)
```

Este atributo cumple una funcion de auditoria basica. Permite identificar cuando fue insertado cada registro y facilita futuras tareas administrativas como validacion de cargas, control de versiones del dataset o comparacion entre distintas ejecuciones.

6.6. Aislamiento logico por fuente

El atributo `dataset_source` no solo documenta el origen del registro, sino que tambien apoya procedimientos administrativos y analiticos. Gracias a este campo es posible:

- confirmar el origen del documento sin depender unicamente del nombre de la coleccion;
- realizar filtros o agregaciones por proveedor del dataset;
- facilitar futuras consolidaciones si en algun momento se decide unificar ambas fuentes en una misma coleccion.

7. Modelo de datos almacenado

7.1. Coleccion buildings_microsoft

```
{
  "geometry": {
    "type": "Polygon o MultiPolygon",
    "coordinates": []
  },
  "area_m2": 123.45,
  "dataset_source": "microsoft",
  "ingested_at": "ISODate"
}
```

7.2. Coleccion buildings_google

```
{
  "geometry": {
    "type": "Polygon o MultiPolygon",
    "coordinates": []
  },
  "area_m2": 123.45,
  "dataset_source": "google",
  "confidence_score": 0.91,
  "ingested_at": "ISODate"
}
```

8. Comandos y procedimientos operativos

8.1. Ejecucion de la aplicacion

```
pip install pymongo geopandas shapely pandas requests
python main.py
```

8.2. Secuencia operativa para la carga

1. ejecutar el programa principal;
2. seleccionar la opcion 3 o 4 en el menu;
3. verificar la disponibilidad del archivo fuente;
4. configurar la direccion de MongoDB;
5. confirmar la carga;
6. esperar la insercion masiva en la coleccion correspondiente.

8.3. Resultado esperado en MongoDB

Al finalizar el procedimiento, la base de datos debe contener:

- una coleccion `buildings_microsoft` o `buildings_google` con footprints cargados;
- un indice geoespacial `2dsphere` sobre el campo `geometry`;
- documentos con sello temporal de ingesta y marca de fuente.

9. Aporte de la semana 3 a la administracion del proyecto

Desde el enfoque de administracion de bases de datos, la semana 3 no solo agrega datos de edificios. Tambien deja preparada la estructura operativa necesaria para la siguiente fase del proyecto:

- colecciones organizadas por dominio de informacion;
- geometrias compatibles con consultas espaciales;
- indices creados para acelerar operaciones posteriores;
- base documental flexible para integrar atributos diferenciados;
- procedimiento reproducible de carga en MongoDB.

10. Conclusion

La entrega de la semana 3 deja implementado un procedimiento completo de carga de footprints en MongoDB y documenta las tareas administrativas asociadas al almacenamiento geoespacial. El sistema ya contempla decisiones importantes de administracion de base de datos, como la separacion por colecciones, la normalizacion espacial, la indexacion `2dsphere`, la insercion masiva y la trazabilidad temporal de los registros.

Con esta base, la informacion de edificios queda preparada para ser utilizada en la semana 4, donde se ejecutaran consultas espaciales y procesos de agregacion sobre municipios PDET.